

CLP Reproduction: Summary

Adaptive multi-token prediction, Qwen3.5-2B, adaptive-mtp toolkit

What we set out to do

We reproduced CLP, a lightweight method from a recent paper, on our own setup. CLP is a small model that looks at a language model's internal state and decides, in the moment, how many extra words the model can safely draft ahead before checking itself. The goal was to follow the paper's method faithfully and see whether it actually works and actually speeds things up, using our own model and our own data, and to compare it against the simpler methods already available in the toolkit we are building this on.

What we built

We built the full pipeline needed to test this end to end. We collected real training data by running the model on real conversations and recording, at many points along the way, what its internal state looked like and whether its own draft-ahead guess turned out right or wrong. We trained a small model on this data to make the same kind of prediction. Then we connected that trained model to real, live text generation, so it could actually control how far the model drafts ahead while it writes, following the paper's own rule: always write the real next word normally, and only draft further ahead when the small model is confident enough to trust.

The comparison

We compared eight approaches on the same set of real, held out prompts: writing text with no drafting at all, four existing methods already in the toolkit (each using a different signal to decide how far to draft), CLP, and two plain fixed methods that always draft the same number of words ahead regardless of context, as a simple baseline.

Method	Speed vs. plain writing	Words per second
Plain writing (no drafting)	1.00x	45.6
Entropy-based	1.30x	59.1
Max-probability-based	1.37x	62.4
Margin-based	1.35x	61.7
History-based	1.42x	64.9
CLP (ours)	1.34x	61.0
Fixed, always 4 ahead	1.72x	78.2
Fixed, always 2 ahead	1.57x	71.5

What we found

CLP works. It produces a real, measurable speedup of about 1.34x over plain writing, and its output text checks out correctly against the same rigorous check we ran for every other method. Among the adaptive

methods (the ones that actually look at context, as opposed to the two fixed baselines), CLP sits in the middle of the pack for speed. The existing history-based method in the toolkit is currently the fastest of the adaptive group. The two fixed baselines that always draft the same number of words every time were faster still, though less reliable, which suggests some of the raw speed on offer here does not strictly require an adaptive method to capture.

Separately, we ran a correctness check across all eight methods, including the two fixed baselines that have no decision-making logic at all. All eight showed occasional cases where the generated text did not exactly match plain writing. Because this happened even for the fixed baselines, with no adaptive logic involved whatsoever, this points to a property of how this particular model interacts with the underlying generation engine, not something caused by any of the methods being compared, including CLP.

A caveat worth flagging

This comparison used ten held out examples, which is the entire test set available to us from our current data source. It is a real, working first result, but a small sample. The relative ordering of the methods, especially the close ones, could shift with more data. A natural next step, if this direction seems worth pursuing further, would be collecting a larger test set before drawing firmer conclusions.

Bottom line

CLP reproduces successfully on our setup: it works, it is correct, and it gives a real speedup. It is not clearly better than the simplest existing method already in the toolkit on this small test set, but it is a solid, working alternative built on a different kind of signal, and the fact that it holds up under the same correctness check as everything else is a meaningful result in itself.